

Sequential Orchestration

Important

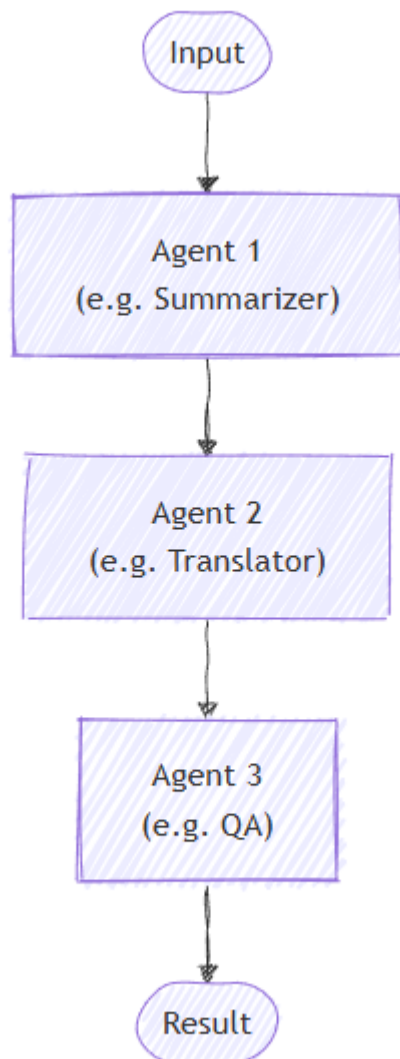
Agent Orchestration features in the Agent Framework are in the experimental stage. They are under active development and may change significantly before advancing to the preview or release candidate stage.

In sequential orchestration, agents are organized in a pipeline. Each agent processes the task in turn, passing its output to the next agent in the sequence. This is ideal for workflows where each step builds upon the previous one, such as document review, data processing pipelines, or multi-stage reasoning.

To learn more about the pattern, such as when to use the pattern or when to avoid the pattern in your workload, see [Sequential orchestration](#).

Common Use Cases

A document passes through a summarization agent, then a translation agent, and finally a quality assurance agent, each building on the previous output:



What You'll Learn

- How to define a sequence of agents, each with a specialized role
- How to orchestrate these agents so that each processes the output of the previous one
- How to observe intermediate outputs and collect the final result

Define Your Agents

Agents are specialized entities that process tasks in sequence. Here, we define three agents: an analyst, a copywriter, and an editor.

💡 Tip

The [`ChatCompletionAgent`](#) is used here, but you can use any [agent type](#).

C#

```
using Microsoft.SemanticKernel;
using Microsoft.SemanticKernel.Agents;
using Microsoft.SemanticKernel.Agents.Orchestration;
using Microsoft.SemanticKernel.Agents.Orchestration.Sequential;
using Microsoft.SemanticKernel.Agents.Runtime.InProcess;

// Create a kernel with an AI service
Kernel kernel = ...;

ChatCompletionAgent analystAgent = new ChatCompletionAgent {
    Name = "Analyst",
    Instructions = "You are a marketing analyst. Given a product description, identify:\n- Key features\n- Target audience\n- Unique selling points",
    Kernel = kernel,
};

ChatCompletionAgent writerAgent = new ChatCompletionAgent {
    Name = "Copywriter",
    Instructions = "You are a marketing copywriter. Given a block of text describing features, audience, and USPs, compose a compelling marketing copy (like a newsletter section) that highlights these points. Output should be short (around 150 words), output just the copy as a single text block.",
    Kernel = kernel,
};

ChatCompletionAgent editorAgent = new ChatCompletionAgent {
    Name = "Editor",
    Instructions = "You are an editor. Given the draft copy, correct grammar, improve clarity, ensure consistent tone, give format and make it polished. Output the final improved copy as a single text block.",
    Kernel = kernel,
};
```

Optional: Observe Agent Responses

You can create a callback to capture agent responses as the sequence progresses via the `ResponseCallback` property.

C#

```
ChatHistory history = [];

ValueTask responseCallback(ChatMessageContent response)
{
    history.Add(response);
    return ValueTask.CompletedTask;
}
```

Set Up the Sequential Orchestration

Create a `SequentialOrchestration` object, passing in the agents and the optional response callback.

C#

```
SequentialOrchestration orchestration = new(analystAgent, writerAgent, editorAgent)
{
    ResponseCallback = responseCallback,
};
```

Start the Runtime

A runtime is required to manage the execution of agents. Here, we use `InProcessRuntime` and start it before invoking the orchestration.

C#

```
InProcessRuntime runtime = new InProcessRuntime();
await runtime.StartAsync();
```

Invoke the Orchestration

Invoke the orchestration with your initial task (e.g., a product description). The output will flow through each agent in sequence.

C#

```
var result = await orchestration.InvokeAsync(
    "An eco-friendly stainless steel water bottle that keeps drinks cold for 24 hours",
    runtime);
```

Collect Results

Wait for the orchestration to complete and retrieve the final output.

C#

```
string output = await result.GetValueAsync(TimeSpan.FromSeconds(20));
Console.WriteLine($"\\n# RESULT: {text}");
Console.WriteLine("\\n\\nORCHESTRATION HISTORY");
foreach (ChatMessageContent message in history)
```

```
{  
    this.WriteAgentChatMessage(message);  
}
```

Optional: Stop the Runtime

After processing is complete, stop the runtime to clean up resources.

C#

```
await runtime.RunUntilIdleAsync();
```

Sample Output

plaintext

```
# RESULT: Introducing our Eco-Friendly Stainless Steel Water Bottles - the perfect  
companion for those who care about the planet while staying hydrated! Our bottles ...
```

ORCHESTRATION HISTORY

```
# Assistant - Analyst: **Key Features:**
```

- Made from eco-friendly stainless steel
- Insulation technology that maintains cold temperatures for up to 24 hours
- Reusable and sustainable design
- Various sizes and colors available (assumed based on typical offerings)
- Leak-proof cap
- BPA-free ...

```
# Assistant - copywriter: Introducing our Eco-Friendly Stainless ...
```

```
# Assistant - editor: Introducing our Eco-Friendly Stainless Steel Water Bottles -  
the perfect companion for those who care about the planet while staying hydrated! Our  
bottles ...
```



Tip

The full sample code is available [here](#)

Next steps

Group Chat Orchestration

Last updated on 07/22/2025